DO NOW — RECALL

Lesson 120 · Following Algorithms and Building Trace Ta

Name: _____ Class: _____ Date: _____

1. What drives decisions in spiral development?

2. Why can an array provide direct indexed access?

3. A full adder receives A=1, B=1 and carry-in=0. What are sum and carry-out?

4. For the binary value 1010, what is its decimal value?

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

Identification, assessment and mitigation of risk.

2. Model answer

Its element locations can be calculated from base address and index.

3. Model answer

Sum 0 and carry-out 1.

4. Model answer

10.

By the end, you can…

• Find variables, data structures, conditions and inputs.

• Update a table only when an assignment or state change occurs.

• Compare produced output against constraints, edge cases and intention.

• Trace recursive calls, identify a base case and compare with iteration.

Vocabulary — know these terms

trace table — Record of algorithm state as steps execute.

iteration — Repeated execution of instructions.

dry run — Manual simulation of algorithm.

boundary test — Test at edge of valid input domain.

recursion — A method calls itself with a smaller or simpler version of the problem.

base case — Stopping condition that returns without another recursive call.

NEW LEARNING

Read, annotate and explain

Identify state

Find variables, data structures, conditions and inputs.

Trace each step

Update a table when an assignment or state change occurs. For a recursive function, record each call , its argument and its eventual return value; unlike a loop, calls wait on the call stack.

Check outcome

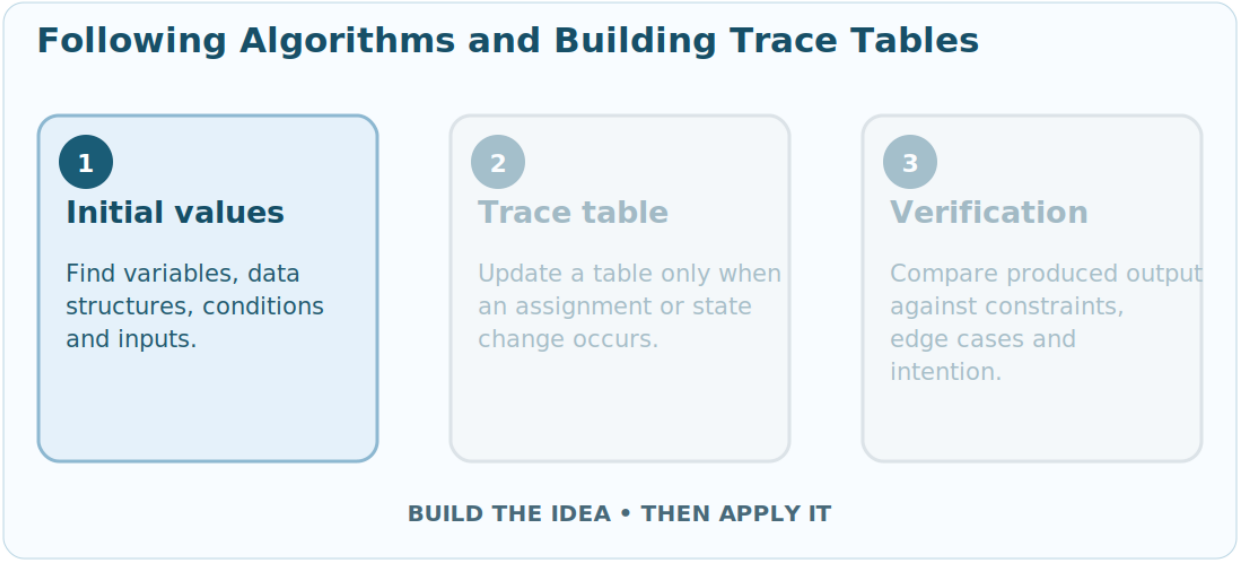
Compare output against constraints and edge cases. Recursion must reach a base case that returns without another recursive call, or it cannot terminate safely.

Explain one step in your own words, using the correct technical terms.

New Learning · original lesson diagrams

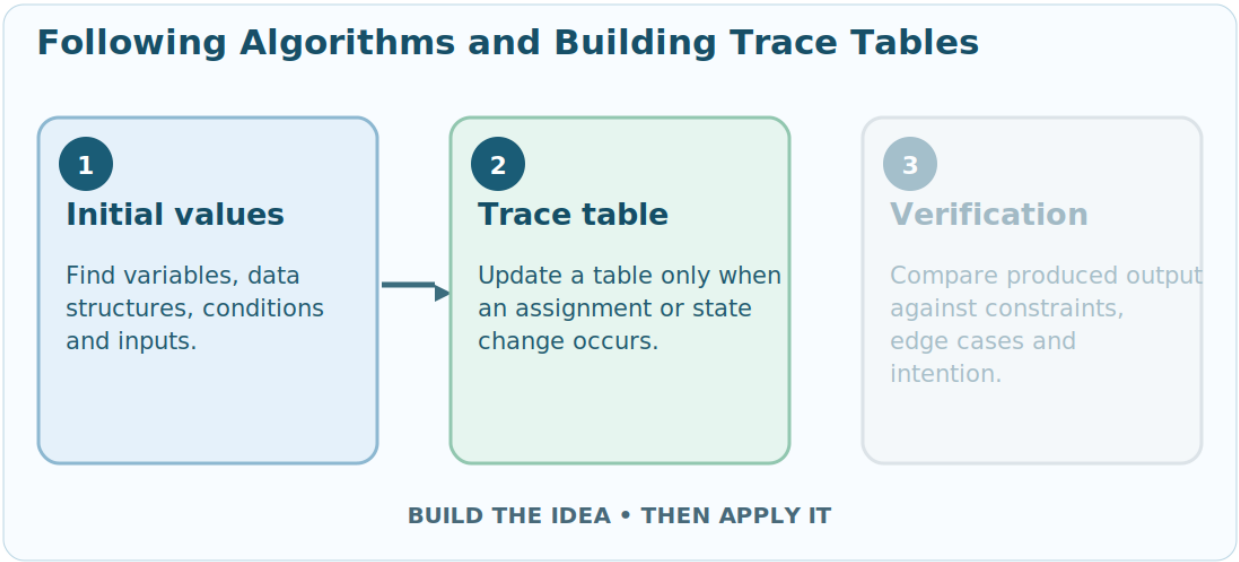
Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



Following Algorithms and Building Trace Tables: Initial values then Trace table then Verification

New Learning

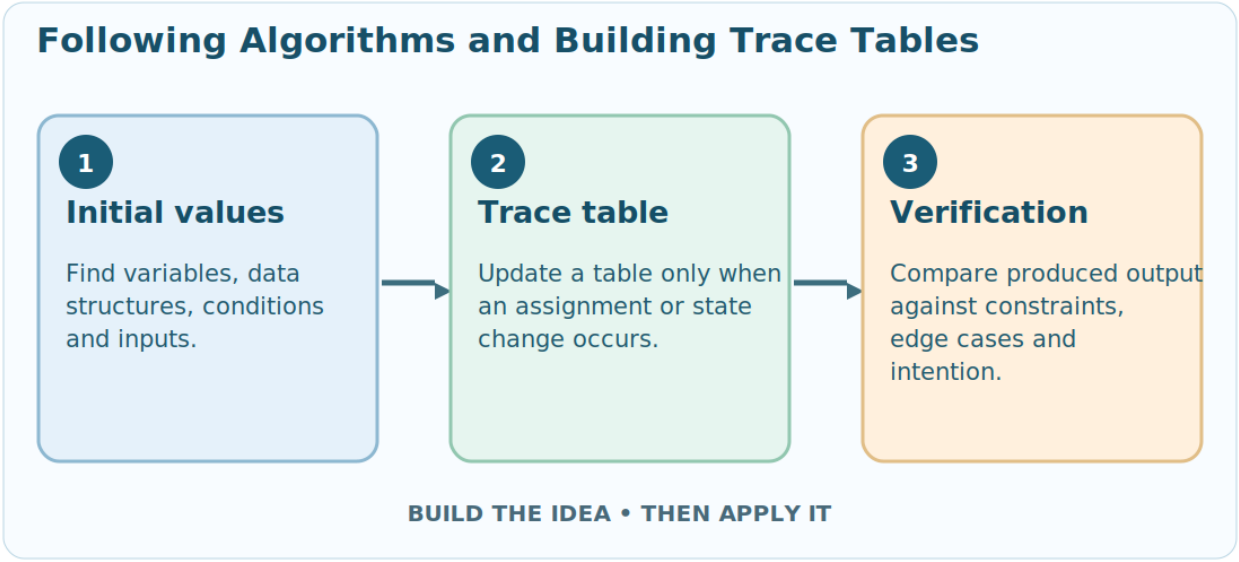


Following Algorithms and Building Trace Tables: Initial values then Trace table then Verification

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



Following Algorithms and Building Trace Tables: Initial values then Trace table then Verification

I DO — WORKED EXAMPLE

Follow the worked method

Trace total ← 0; FOR n = 1 TO 4: IF n MOD 2 = 0 THEN total ← total + n; NEXT n; OUTPUT total.

Model answer

At n=1 total 0; n=2 total 2; n=3 total 2; n=4 total 6; output 6. Record n and total at each loop step.

Check: link each claim to the scenario and use precise OCR terminology.

Spot the stopping condition

n == 0 is the base case : return 0 immediately. Otherwise SumTo(n - 1) invokes the same method with a smaller argument. Assume n is non-negative.

Trace calls before returns

SumTo(3) waits for SumTo(2) , then SumTo(1) , then SumTo(0) . The first three calls remain active until the base case returns.

Follow the returns in reverse order

SumTo(0)=0 ; then SumTo(1)=1 , SumTo(2)=3 , SumTo(3)=6 . An iterative alternative would use a loop and running total; recursion instead retains pending calls.

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Trace total ← 0; FOR n = 1 TO 4: IF n MOD 2 = 0 THEN total ← total + n; NEXT n; OUTPUT total.

At n=1 total 0; n=2 total 2; n=3 total 2; n=4 total 6; output 6. Record n and total at each loop step.



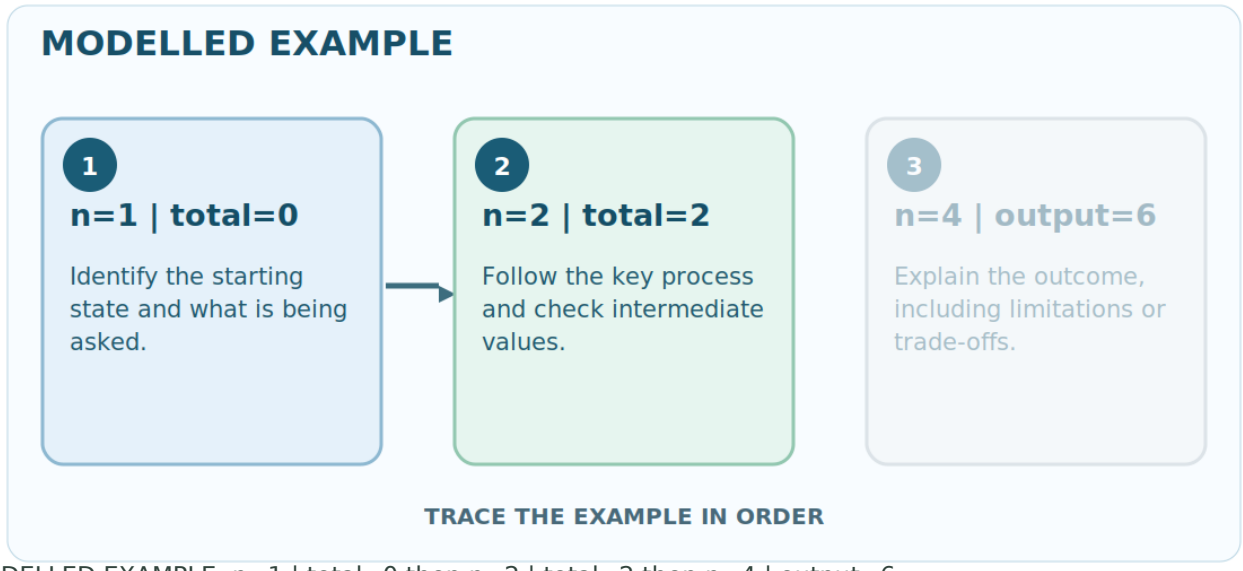
MODELLED EXAMPLE: n=1 | total=0 then n=2 | total=2 then n=4 | output=6

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Trace total ← 0; FOR n = 1 TO 4: IF n MOD 2 = 0 THEN total ← total + n; NEXT n; OUTPUT total.

At n=1 total 0; n=2 total 2; n=3 total 2; n=4 total 6; output 6. Record n and total at each loop step.



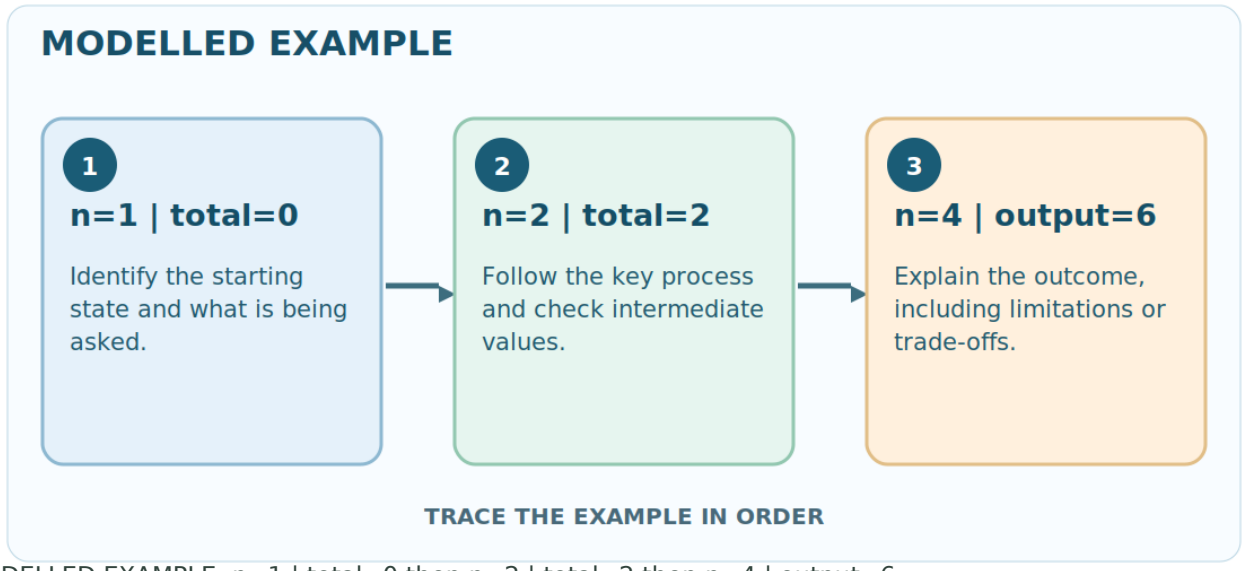
MODELLED EXAMPLE: n=1 | total=0 then n=2 | total=2 then n=4 | output=6

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Trace total ← 0; FOR n = 1 TO 4: IF n MOD 2 = 0 THEN total ← total + n; NEXT n; OUTPUT total.

At n=1 total 0; n=2 total 2; n=3 total 2; n=4 total 6; output 6. Record n and total at each loop step.



MODELLED EXAMPLE: n=1 | total=0 then n=2 | total=2 then n=4 | output=6

WE DO — GUIDED PRACTICE

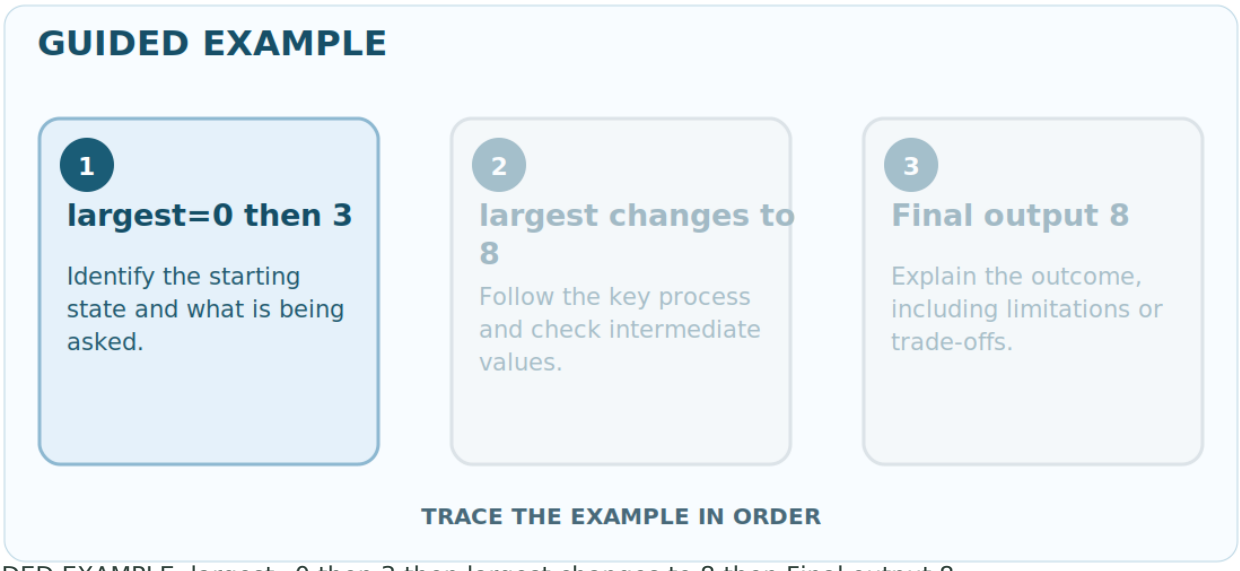
Try → Discuss → Improve

We Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Trace largest ← 0; values=[3,8,5]; FOR each value IF value>largest THEN largest←value; OUTPUT largest.

Identify the inputs, relevant facts and exact output or judgement required.



GUIDED EXAMPLE: largest=0 then 3 then largest changes to 8 then Final output 8

WE DO — TRY · DISCUSS · IMPROVE

TRY: Use the original guided questions and diagrams in the preceding We Do pages. DISCUSS: Explain and challenge methods before independent practice. IMPROVE: Note one change now; compare with the source model only AFTER the Exit Ticket.

Guided 1: What must we decide? Identify the inputs, relevant facts and exact output or judgement required.

DISCUSS — Compare your methods. Justify one important decision, identify one difference, and agree a specific improvement.

IMPROVE — Write one precise correction or stronger explanation in your own answer.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Trace count ← 0; FOR i = 1 TO 5: IF i MOD 2=1 THEN count←count+1; NEXT; OUTPUT count. [6]

Question 2

Explain why initialising max ← 0 can cause a fault when finding the maximum of negative values. [6]

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

Trace recursive SumTo(4) using the method shown, name its base case and explain one difference from an iterative solution. [8]

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

A trace table only needs the final value of each variable.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

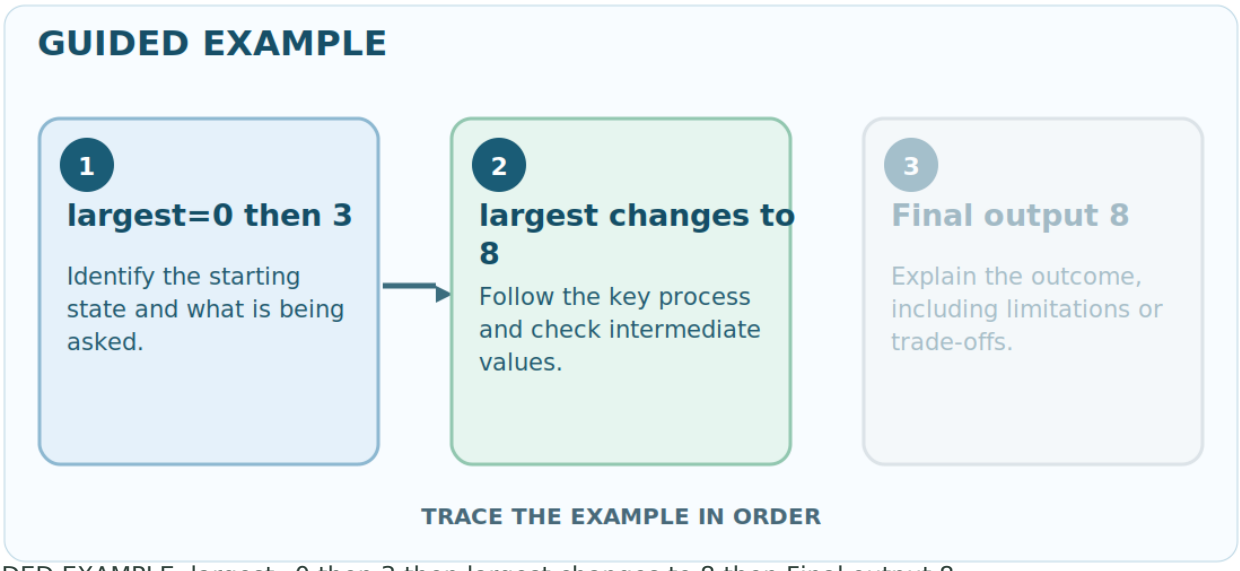
Next step: revisit the model or a relevant revision-guide section if needed.

We Do Check · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Trace largest ← 0; values=[3,8,5]; FOR each value IF value>largest THEN largest←value; OUTPUT largest.

Identify the inputs, relevant facts and exact output or judgement required.



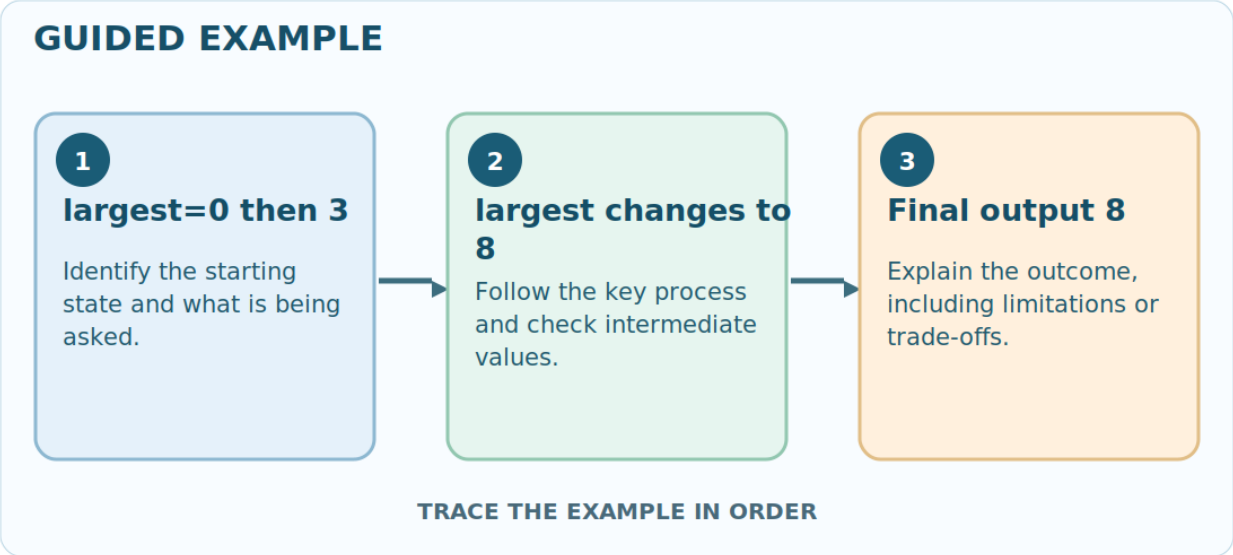
GUIDED EXAMPLE: largest=0 then 3 then largest changes to 8 then Final output 8

We Do Check · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Trace largest ← 0; values=[3,8,5]; FOR each value IF value>largest THEN largest←value; OUTPUT largest.

Identify the inputs, relevant facts and exact output or judgement required.



GUIDED EXAMPLE: largest=0 then 3 then largest changes to 8 then Final output 8

WE DO — SOURCE CHECK / IMPROVE

Only use this section AFTER attempting the We Do, You Do and Exit Ticket. Compare with the original classroom lesson's CHECK explanation. Correct errors in GREEN PEN and explain why.

Guided 1 — original CHECK

Work through the method

Use the middle stage of the visual to follow the process and state each intermediate result.

Shared worked answer

Progress: initial 0; after 3 → 3; after 8 → 8; after 5 stays 8; final output 8. The initial zero fails if all values are negative; use first element instead.

IMPROVE: Explain one correction to your own reasoning.